

PALANTIR FOUNDRY PROJECT ENABLEMENT PROGRAM

Day 2 Study Material

Module 1: OSDK Architecture

Phase 1 — Foundation Accelerator

Author: Rajesh Pasham

Table of Contents

Table of Contents	2
1. Session Overview & Objectives	3
2. What Is the Ontology SDK (OSDK)?	4
2.1 Where the OSDK Sits.....	4
2.2 Why Not Just Call the REST API Directly?	4
3. What's Inside a Generated OSDK Package	5
4. Developer Console	6
4.1 Creating Your First Application	6
5. Authentication	7
5.1 Public OAuth Client	7
5.2 Confidential OAuth Client.....	7
5.3 Token-Based Authentication	7
6. Object APIs — Reading Data.....	8
6.1 Pagination Deep Dive	8
6.2 Object Instance Anatomy	8
7. Error Handling.....	9
8. Actions & Queries — Writing Data.....	10
8.1 Queries vs. Actions at a Glance	10
9. Choosing Between TypeScript and Python	11
10. Security Best Practices.....	12
11. Key Terms — New This Session.....	13
12. Frequently Asked Questions.....	14
13. Today's Definition of Done	15
14. Reference Documentation.....	16
15. What's Next — Day 3 Preview	17
Module 2: React Application Development.....	17

1. Session Overview & Objectives

Day 1 gave you a shared architectural picture of Foundry and a working Ontology (Vehicle and Transponder). Day 2 puts that Ontology in front of code for the first time, through the Ontology SDK (OSDK).

By the end of today's session, you will be able to:

- Explain what the OSDK is and where it sits in the Foundry architecture
- Create a Developer Console application and generate an OSDK package
- Describe the three common authentication patterns and when to use each
- Read, filter, and sort Ontology objects from TypeScript or Python code
- Explain how Action Types are invoked from the OSDK, and how Queries differ from Actions
- Handle pagination and errors correctly in OSDK calls
- Build and run your first OSDK application against your own Ontology

2. What Is the Ontology SDK (OSDK)?

The OSDK is a generated, typed client library that gives application code ergonomic access to your Ontology. It is available for TypeScript, Python, and Java, with OpenAPI specifications available for any other language.

Two ideas make the OSDK distinctive:

- **It is generated, not generic:** the OSDK is built specifically for your Ontology. If you have a Vehicle object type, the OSDK gives you a typed Vehicle class with methods to query and act on it
- **It stays in sync:** the OSDK is regenerated whenever your Ontology's object types, links, or actions change, so a mismatch between your code and your Ontology is caught by the type system rather than at runtime

2.1 Where the OSDK Sits

Your Application → OSDK Client (generated) → Platform SDK (generic APIs) → Ontology (Objects & Actions)

Platform SDKs are generic libraries that work against any Foundry enrollment. OSDKs add a further, ergonomic layer of abstraction on top, scoped to your specific object types and actions.

2.2 Why Not Just Call the REST API Directly?

Foundry's platform APIs are available as raw REST endpoints, and nothing stops an application from calling them directly. The OSDK exists because a typed layer on top pays for itself quickly: property names, data types, and action parameters are all checked at compile time, editor autocomplete replaces guessing endpoint URLs and payload shapes, and a schema change surfaces as a build error instead of a production incident.

3. What's Inside a Generated OSDK Package

A generated SDK package is organized predictably, which makes it easy to find what you need once you know the shape:

- **objects/**: one typed class per object type in your application — Vehicle, Transponder
- **actions/**: one typed function per action type — markVehicleForMaintenance
- **queries/**: one typed function per query, once your application defines any

The client.ts file that wires authentication to these generated types is code you write yourself, not something the generator produces — that separation keeps your credentials and configuration out of generated, regeneratable code.

4. Developer Console

Developer Console is the application used to create and manage OSDK applications and OAuth clients. A Custom Application in Developer Console bundles:

- An OAuth client (public or confidential)
- A generated OSDK package, scoped to the Ontology resources you select
- Optional web hosting for frontend-only applications

4.1 Creating Your First Application

1. In Developer Console, select + New application.
2. Choose Client-facing application for a browser-based app.
3. On the Permissions page, use the Authorization code grant and set a redirect URL, e.g. `http://localhost:8080/auth/callback`.
4. Add the object types, link types, and action types your application needs (Vehicle and Transponder, for today).
5. Foundry generates an SDK package — install instructions and API documentation (TypeScript, Python, and cURL) are shown in the application's SDK tab.

5. Authentication

Every OSDK client needs an authentication method. Three patterns cover most cases, and today's exercise uses two of them.

5.1 Public OAuth Client

For browser applications where a human user logs in interactively. Foundry handles the login flow (Authorization Code with PKCE) and redirects back to your application's redirect URL with a session established. A public client has no secret to protect — that is precisely why it is safe to ship inside browser-delivered code.

5.2 Confidential OAuth Client

For server-to-server integrations with no user present. A client ID and client secret are exchanged directly for an access token. Client credentials applications are well suited to backend services, automated systems, and API integrations with external platforms — this is the pattern later sprints use for scheduled polling against UPS and Neology.

5.3 Token-Based Authentication

The simplest option for local scripts, notebooks, and CLIs: a long-lived user or service token stands in for a session, with no OAuth redirect flow at all. This is the fastest way to get a script running, and is what today's Python quick-start uses. The trade-off is that a token is as powerful and as sensitive as a password, and needs the same handling discipline.

Security: Keep OAuth client secrets and tokens out of source control. Use environment variables or a secrets manager, never a hard-coded string in a committed file. Add `.env` to `.gitignore` before your first commit.

6. Object APIs — Reading Data

The generated OSDK exposes a small, consistent set of methods for reading objects:

Method	Purpose
<code>client(Vehicle).fetchPage()</code>	Read one page of Vehicle objects
<code>.where({ status: { \$eq: "Active" } })</code>	Filter objects by a property value
<code>.fetchPage({ \$orderBy: { year: "desc" } })</code>	Sort results
<code>.iterate()</code>	Stream through every matching object, across pages

Every method is generated against your object type's real properties, so your editor autocompletes property names such as `vehicleId`, `make`, and `status`, and rejects a typo at compile time rather than at runtime.

6.1 Pagination Deep Dive

`fetchPage()` returns one page of results plus a `nextPageToken` when more results exist beyond that page. Passing that token back into a subsequent `fetchPage({ $nextPageToken })` call retrieves the next page. The `.iterate()` method wraps this loop for you internally, yielding every matching object one at a time regardless of how many pages that spans.

As a rule of thumb: use `fetchPage()` when a UI needs to control exactly how many rows to show at once (a paged table), and use `iterate()` for background jobs or scripts that need to process every matching object.

6.2 Object Instance Anatomy

Every object instance the OSDK returns exposes your defined properties directly (`vehicle.make`, `vehicle.status`), plus a small set of OSDK-added metadata fields:

- **\$primaryKey**: the object's primary key value, without needing to know which property holds it
- **\$title**: the object's display title, as configured on the object type
- **\$apiName**: which object type this instance came from — useful in generic, reusable UI code

7. Error Handling

OSDK calls can fail for the same reasons any network call can: permissions, validation, connectivity, or a malformed request. Wrapping a call in try/catch and checking whether the error is an OSDK-specific error (via the `isOsdError` helper exported from `@osdk/client`) lets you handle expected failures — like a permissions error or a validation error — differently from unexpected ones, which you generally want to rethrow rather than silently swallow.

An OSDK error typically carries an `errorName` and a set of parameters describing what went wrong, which is usually enough detail to fix a broken action call without needing to inspect a raw network response.

8. Actions & Queries — Writing Data

Every write to the Ontology happens through an Action Type — never a direct edit of an object. This is true whether the write comes from Workshop, a Function, or an OSDK application.

Actions are generated into the OSDK the same way objects are: with typed parameters and a typed result. Today, you will define a simple action, `markVehicleForMaintenance`, that sets a `Vehicle`'s status property, and call it from code.

Queries are read-only, reusable server-side logic — useful for aggregations or lookups that are awkward to express as a client-side filter. We will use Queries more heavily from Week 2 onward.

8.1 Queries vs. Actions at a Glance

	Query	Action
Purpose	Read-only computation	Write to the Ontology
Changes data?	Never	Always, via a Function
Typical use	Aggregations, complex lookups	Create / update / delete an object
Called from OSDK as	<code>client(myQuery).executeFunction()</code>	<code>client(myAction).applyAction()</code>

Important: The exact method name and parameter shape for an action are shown in your application's generated API Documentation in Developer Console — always treat that as the source of truth over any example in this document.

9. Choosing Between TypeScript and Python

	TypeScript OSDK	Python OSDK
Best for	Browser and Node.js applications	Scripts, notebooks, data workflows
Auth pattern used today	Public OAuth client	Token-based (UserTokenAuth)
Package manager	npm / pnpm	pip / conda

Today's core exercise uses TypeScript, since it is the path the React application (starting Day 3) will build on. A Python quick-start is included as a stretch goal for anyone who prefers scripting.

10. Security Best Practices

- Never commit a client secret or a personal access token to source control
- Load secrets from environment variables or a secrets manager, never as a hard-coded string
- Prefer a public OAuth client for anything shipped to a browser — it has no secret to leak
- Rotate a token or secret immediately if it is ever committed or shared by accident
- Add .env (or your platform's equivalent) to .gitignore before your first commit

11. Key Terms — New This Session

Term	Definition
OSDK	Ontology Software Development Kit — a generated, typed client library for your Ontology.
Developer Console	The application used to create and manage OSDK applications and OAuth clients.
Client ID	A public identifier for an OAuth client, used in the authentication flow.
Redirect URL	The URL Foundry sends the browser back to after a successful login.
Ontology RID	The unique resource identifier of your Ontology, required when creating an OSDK client.
Action Type	Defines how an object can be created, updated, or deleted; the only path for writes.
Query	Read-only, reusable server-side logic exposed through the Ontology.
fetchPage	The core OSDK method for reading a page of objects, optionally filtered and sorted.
nextPageToken	An opaque token returned by fetchPage(), passed back to retrieve the next page.
\$primaryKey / \$title / \$apiName	OSDK-added metadata fields present on every fetched object instance.
isOsdError	A helper from @osdk/client used to distinguish OSDK-specific errors from other exceptions.

12. Frequently Asked Questions

- **The browser login loop never completes.:** Double-check your redirect URL matches exactly what is configured in Developer Console, including the port.
- **My action call is rejected.:** Re-check the exact, case-sensitive parameter names in your generated API Documentation.
- **TypeScript can't find my generated package.:** Re-run the install command shown on your application's SDK tab — it is specific to your application and changes when you add new resources.
- **Should I use fetchPage() or iterate()?:** Use fetchPage() when a UI needs to control how many rows to show at once; use iterate() for scripts or jobs that need every matching object.

13. Today's Definition of Done

You have completed today's session when:

- A Developer Console application exists, scoped to Vehicle, Transponder, and markVehicleForMaintenance
- Your local TypeScript project authenticates successfully via the public OAuth client
- Running your script prints all Active vehicles, sorted by year
- Running your script prints all in-stock transponders
- Calling sendToMaintenance updates a real Vehicle's status, visible in Ontology Manager
- (Stretch) The Python script reproduces the vehicle read using token-based authentication

14. Reference Documentation

Official Palantir Foundry documentation pages used as the basis for today's material:

- Ontology SDK — Overview: palantir.com/docs/foundry/ontology-sdk/overview
- Ontology SDK — Create a new OSDK application: palantir.com/docs/foundry/ontology-sdk/create-a-new-osdk
- Ontology SDK — Developer Console navigation: palantir.com/docs/foundry/ontology-sdk/navigation
- Ontology SDK — Python OSDK: palantir.com/docs/foundry/ontology-sdk/python-osdk
- Developer Console — Overview: palantir.com/docs/foundry/developer-console/overview
- Developer Console — Add OSDK to an existing application: palantir.com/docs/foundry/developer-console/how-to-add-to-existing-typescript
- Consumer mode — Foundry-hosted OAuth applications: palantir.com/docs/foundry/consumer-mode/oauth-application-setup
- Consumer mode — Client credentials applications: palantir.com/docs/foundry/consumer-mode/client-credentials-setup
- API Reference — Software development kit (SDK) overview: palantir.com/docs/foundry/api/general/overview/sdks

Keeping current: OSDK package names, exact method signatures, and Developer Console screens evolve frequently. Always cross-check against your application's generated API Documentation and the in-platform help before each session.

15. What's Next — Day 3 Preview

Module 2: React Application Development

- React architecture, components and hooks
- Routing and responsive layout
- State management
- Material UI

Hands-on lab: develop a mobile-responsive application.